

LABORATORIO 13:

Compresión y Descompresión de Archivos mediante Algoritmo de Huffman

1. Introducción y Fundamentos Teóricos

El algoritmo de Huffman es un método de compresión de datos sin pérdida (*loss/less*) desarrollado por David Huffman en 1952.

El algoritmo asigna códigos binarios de longitud variable a los caracteres de una fuente, de forma que los caracteres más frecuentes reciben códigos más cortos y los menos frecuentes códigos más largos, respetando la propiedad de prefijo (ningún código es prefijo de otro).

Esto permite una compresión sin pérdida: el texto original puede recuperarse exactamente a partir del bitstream comprimido.

Se utiliza un enfoque codicioso (*greedy*) para asignar códigos de longitud variable a los caracteres de un texto, basándose en sus frecuencias de aparición.

Los caracteres más frecuentes reciben códigos más cortos, mientras que los menos frecuentes reciben códigos más largos, cumpliendo con la **propiedad de prefijo** (ningún código es prefijo de otro).

1.1. Ecuaciones de Evaluación

Para medir la eficiencia del algoritmo, se utilizarán las siguientes métricas estandarizadas (Derivadas del concepto de entropía de Shannon estudiado en Clases):

- **Entropía de la Fuente (H):** Define el límite teórico de compresión en bits por carácter.

$$H = - \sum_{i=1}^n P(c_i) \log_2 P(c_i)$$

Donde $P(c_i)$ es la probabilidad (frecuencia relativa) del carácter c_i .

- **Longitud Media del Código ($\bar{L}$):**

$$\bar{L} = \sum_{i=1}^n P(c_i) \cdot |b_i|$$

Donde $|b_i|$ es la longitud en bits del código asignado al carácter c_i .

- **Tasa de Compresión (CR - Compression Ratio):**

$$CR = \frac{\text{Tamaño del Archivo Original (bytes)}}{\text{Tamaño del Archivo Comprimido (bytes)}}$$

- **Porcentaje de Ahorro (SP - Space Saving):**

$$SP = \left(1 - \frac{\text{Tamaño del Archivo Comprimido}}{\text{Tamaño del Archivo Original}}\right) \times 100\%$$

2. Estructuras de Datos Requeridas en C

Los estudiantes deberán definir, como mínimo, las siguientes estructuras para la representación del árbol de Huffman y la cola de prioridad:

C/C++

```
// Estructura para los nodos del árbol de Huffman
typedef struct NodoHuffman {
    unsigned char caracter;
    unsigned int frecuencia;
    struct NodoHuffman *izq;
    struct NodoHuffman *der;
} NodoHuffman;

// Estructura para la Cola de Prioridad (Min-Heap)
typedef struct ColaPrioridad {
    unsigned int capacidad;
    unsigned int tamano;
```

```
NodoHuffman **arreglo;  
} ColaPrioridad;
```

3. Especificación del Formato de Archivo Comprimido (.huff)

Para que el archivo sea portable y descompresible, se debe definir un formato binario estricto para la cabecera (*header*). El archivo de salida estructurado constará de:

Sección	Tamaño	Descripción
Cantidad de Caracteres (N)	1 Byte	Número de entradas en la tabla de frecuencias (0 representa 256).
Tabla de Frecuencias	$N \times (1 \text{ Byte} + 4 \text{ Bytes})$	Parejas [Carácter (char)][Frecuencia (uint32_t)] para reconstruir el árbol.
Bits de Relleno (<i>Padding</i>)	1 Byte	Número de bits basura (0 a 7) añadidos al final del bitstream para completar el último byte.
Bitstream Comprimido	Variable	El texto original codificado en bits.

4. Algoritmos y Pseudocódigo

La práctica se divide en cuatro fases principales:

4.1. Fase 1: Construcción del árbol de Huffman

Se usa una cola de prioridad mínima para fusionar repetidamente los dos nodos con menor frecuencia.

None

```
Procedimiento CrearArbolHuffman(tabla_frecuencias)
    cola = CrearColaPrioridadVacía()

    Para cada carácter c con frecuencia f > 0 en tabla_frecuencias:
        nodo = CrearNuevoNodo(c, f, NULL, NULL)
        InsertarColaPrioridad(cola, nodo)

    Mientras Tamaño(cola) > 1:
        izq = ExtraerMinimo(cola)
        der = ExtraerMinimo(cola)

        // Nodo interno sin carácter significativo
        padre = CrearNuevoNodo(0, izq.frecuencia + der.frecuencia,
        izq, der)
        InsertarColaPrioridad(cola, padre)

    Retornar ExtraerMinimo(cola) // Raíz del árbol de Huffman
FinProcedimiento
```

4.2. Fase 2: Generación del diccionario de códigos

Se realiza un recorrido en profundidad (DFS) para asignar códigos binarios a cada hoja.

None

```
Procedimiento GenerarCodigos(nodo, codigo_actual, diccionario)
    Si nodo es Hoja entonces
        diccionario[nodo.caracter] = Clonar(codigo_actual)
        Retornar
    FinSi
    GenerarCodigos(nodo.izq, codigo_actual + "0", diccionario)
    GenerarCodigos(nodo.der, codigo_actual + "1", diccionario)
FinProcedimiento
```

Implementación sugerida en C:

- diccionario puede ser un arreglo de char* de tamaño 256.
- codigo_actual puede ser un buffer dinámico o un arreglo fijo con longitud máxima igual a la altura del árbol.

4.3. Fase 3: Proceso de compresión y escritura binaria

El manejo de bits en C requiere el uso de un búfer de 8 bits (unsigned char) y operadores a nivel de bits (<< , |).

None

```
Procedimiento ComprimirTexto(archivo_entrada, archivo_salida,
diccionario, tabla_frecuencias)
    EscribirCabecera(archivo_salida, tabla_frecuencias)

    buffer_byte = 0
    contador_bits = 0

    Mientras no sea FinDeArchivo(archivo_entrada):
        caracter = LeerCaracter(archivo_entrada)
        codigo = diccionario[caracter]

        Para cada bit en codigo:
            Si bit == '1' entonces
                buffer_byte = buffer_byte | (1 << (7 -
contador_bits))
            FinSi

            contador_bits = contador_bits + 1

            Si contador_bits == 8 entonces
                EscribirByte(archivo_salida, buffer_byte)
                buffer_byte = 0
                contador_bits = 0
            FinSi
        FinPara
    FinMientras

    // Manejo del último byte incompleto (padding)
    Si contador_bits > 0 entonces
        EscribirByte(archivo_salida, buffer_byte)
        bits_relleno = 8 - contador_bits
        ActualizarBitsDeRellenoEnCabecera(archivo_salida,
bits_relleno)
```

```
    Sino
        bits_relleno = 0
        ActualizarBitsDeRellenoEnCabecera(archivo_salida,
bits_relleno)
    FinSi
FinProcedimiento
```

4.4. Fase 4: Proceso de descompresión

None

```
Procedimiento DescomprimirArchivo(archivo_comprimido,
archivo_destino)
    tabla_frecuencias, bits_relleno =
LeerCabecera(archivo_comprimido)
    raiz = CrearArbolHuffman(tabla_frecuencias)
    nodo_actual = raiz

    Mientras haya bytes por leer en archivo_comprimido:
        byte = LeerByte(archivo_comprimido)
        limite_bits = (EsUltimoByte(byte)) ? (8 - bits_relleno) : 8

        Para i desde 0 hasta limite_bits - 1:
            bit = (byte >> (7 - i)) & 1

            Si bit == 0 entonces
                nodo_actual = nodo_actual.izq
            Sino
                nodo_actual = nodo_actual.der
            FinSi

            Si EsNodoHoja(nodo_actual) entonces
                EscribirCaracter(archivo_destino,
nodo_actual.caracter)
                nodo_actual = raiz
            FinSi
        FinPara
    FinMientras
FinProcedimiento
```


5. Instrucciones y Requerimientos de la Práctica

El estudiante deberá entregar una solución modularizada que cumpla con los siguientes hitos de ingeniería de software:

5.1. Módulos requeridos

- `huffman_tree.h / .c`
- Creación y destrucción de nodos.
- Construcción del árbol a partir de la tabla de frecuencias.
- Generación de códigos (DFS).
- `priority_queue.h / .c`
- Implementación del Min-Heap: insertar, extraer mínimo, `heapify`.
- `compressor.h / .c`
- Lectura del archivo de entrada y conteo de frecuencias.
- Escritura de cabecera `.huff`.
- Empaquetado de bits y escritura del bitstream.
- Lectura de cabecera y reconstrucción de tabla de frecuencias.
- Desempaquetado de bits y escritura del archivo destino.
- `main.c`
- Interfaz de línea de comandos (CLI).
- Parseo de flags `-c` (comprimir) y `-d` (descomprimir).
- Orquestación de llamadas a los módulos anteriores.

Notas:

Interfaz por Línea de Comandos: El programa ejecutable debe compilarse mediante un archivo `Makefile` y operar bajo la siguiente sintaxis:

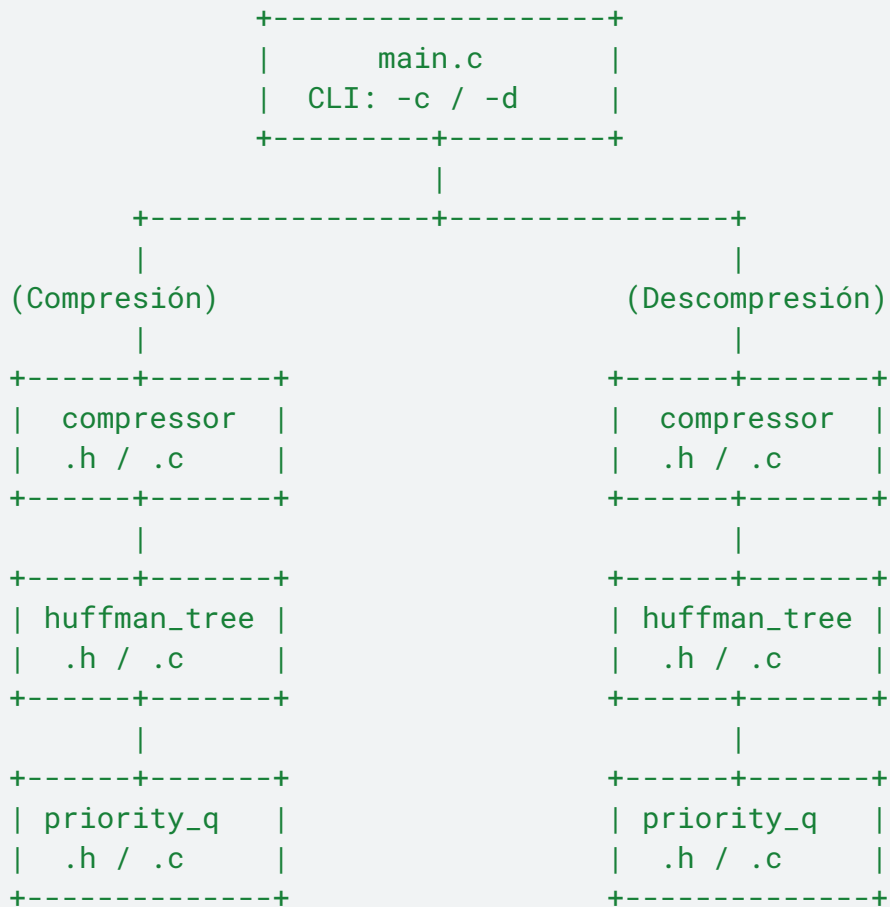
- Compresión: `./huffman -c archivo.txt archivo.huff`
- Descompresión: `./huffman -d archivo.huff archivo_recuperado.txt`

Restricciones de Implementación:

- No está permitido el uso de memoria global estructurada para el flujo de datos. Toda la memoria dinámica asignada con `malloc()` debe ser liberada correctamente mediante `free()` sin dejar fugas (*memory leaks*).
- El búfer de bits debe procesarse estrictamente mediante operaciones de desplazamiento binario.

5.2 Diagrama de la arquitectura lógica

None



1. Flujo de compresión:

- `main.c` detecta `-c` archivo.txt archivo.huff.
- `compressor.c` lee archivo.txt, construye tabla de frecuencias.
- `huffman_tree.c` construye el árbol usando `priority_queue.c`.
- `huffman_tree.c` genera el diccionario de códigos.
- `compressor.c` escribe cabecera y bitstream en archivo.huff.

2. Flujo de descompresión:

- `main.c` detecta `-d` archivo.huff archivo_recuperado.txt.

- b. compressor.c lee cabecera y tabla de frecuencias.
- c. huffman_tree.c reconstruye el árbol con priority_queue.c.
- d. compressor.c recorre el árbol según los bits del bitstream y escribe archivo_recuperado.txt.

6. Interfaz por línea de comandos y Makefile

6.1. Sintaxis de ejecución

Compresión:

```
./huffman -c archivo.txt archivo.huff
```

Descompresión:

```
./huffman -d archivo.huff archivo_recuperado.txt
```

6.2. Makefile (esquema sugerido)

None

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11
OBJ = main.o huffman_tree.o priority_queue.o compressor.o
huffman: $(OBJ)
    $(CC) $(CFLAGS) -o huffman $(OBJ)
main.o: main.c huffman_tree.h priority_queue.h compressor.h
huffman_tree.o: huffman_tree.c huffman_tree.h priority_queue.h
priority_queue.o: priority_queue.c priority_queue.h
compressor.o: compressor.c compressor.h huffman_tree.h
priority_queue.h

clean:
    rm -f *.o huffman
```

7. Implementación y buenas prácticas

Memoria dinámica:

No usar memoria global para el flujo de datos.

- Toda memoria asignada con malloc debe liberarse con free.
- Implementar funciones de destrucción:
- void destruir_arbol(NodoHuffman *raiz);
- void destruirCola(ColaPrioridad *cola);
- void liberar_diccionario(char **diccionario);

Procesamiento de bits:

- Usar estrictamente operaciones de desplazamiento (<<, >>) y OR (|) para empaquetar bits.
- Evitar manipulación de bits mediante strings en tiempo de escritura; las cadenas se usan solo como representación lógica de códigos.

Casos borde recomendados (cada equipo deberá realizar su set de pruebas para este laboratorio):

- Archivo vacío.
- Archivo con un solo carácter repetido.
- Archivo con todos los caracteres distintos.
- Frecuencias iguales (verificar determinismo del heap).